PROJECT WORKFLOW & ARCHITECTURE

ArchitectPoint

How a click in the Angular browser app becomes a row in MySQL — every layer, every file, and four complete request traces with real payloads.

FRONTEND	Angular 16 · TypeScript · port 8081
BACKEND	Spring Boot 3.0.1 · Java 17 · port 8080
DATABASE	MySQL · schema `appdb`
AUTH	Stateless JWT (HS256) · roles `Admin` / `User`
GRADING	24 automated tests — 15 JUnit + 9 Karma
GENERATED	2 October 2026

— Contents

1 **The project in one page**
What it does, who the two actors are, what the twelve endpoints are for

2 **System architecture**
The three tiers and what lives in each

3 **The data model**
Four tables, two foreign keys, and why the relations are unidirectional

4 **The backend, layer by layer**
Filter chain → controller → service → repository → Hibernate

5 **Security: the JWT lifecycle**
How a token is minted, read back, and turned into a 200 or a 403

6 **The frontend, layer by layer**
Routes → guard → component → service → HttpClient

7 **Worked example A — Login**
From the form submit to two different landing pages

8 **Worked example B — User submits an inquiry**
The full happy path, including id-stub resolution and the INSERT

9 **Worked example C — Admin lists all inquiries**
A read path with a JPA derived query and a nested JSON response

10 **Worked example D — A 403 in slow motion**
What happens when a User POSTs an architect

11 **Complete API reference**
Every endpoint, its roles, its payload, and the code that serves it

12 **Screen map**
Sixteen screens, which role reaches them, which service each calls

13 **The grading harness**
How the 24 tests run and what each one asserts

14 **Running it & the traps**
Commands, and the symptom → cause table

1 The project in one page

ArchitectPoint is a two-sided directory. Admins curate a catalogue of architects; users browse it, send inquiries about projects, and leave feedback. Everything else in the codebase exists to serve those three nouns.

The three nouns

ENTITY	WHAT IT IS	WHO CREATES IT	WHO READS IT
Architect	A catalogue listing — name, expertise, location, rating, portfolio link, status	Admin only	Both roles (the list); Admin only (one by id)
ArchitectInquiry	A user's enquiry to one architect — message, priority, contact details, status	User only	Admin sees all; User sees only their own
Feedback	A user's review — text, date, category, optionally tied to an architect	User only	Admin sees all; User sees only their own

The key idea: roles invert per resource

This is the single most important thing to understand about the project, and the thing that surprises everyone reading it for the first time. There is no "Admin can do everything" rule. Permission is assigned *per endpoint*, and it flips depending on the resource:

- **Architects are Admin territory.** Only an Admin may create, read-by-id, update or delete one. A User may only *list* them.
- **Inquiries and feedback are User territory.** Only a User may create one — an Admin POSTing an inquiry gets 403. An Admin may read the whole collection and update an inquiry's status, but may *not* read the per-user endpoints `/user/{id}`, which are reserved for the owning User.

So an Admin is not a superset of a User. They are two disjoint permission sets that overlap on exactly one endpoint: `GET /api/architects`.

> **WHY THIS MATTERS FOR THE CODE**
>
> Because permission is per-URL-and-method and not per-role-hierarchy, all of it is declared in one place — the `authorizeHttpRequests` block of `SecurityConfig.java`. There are no `@PreAuthorize` annotations anywhere in the project, and adding any would create a second, conflicting source of truth.

How a feature is built, end to end

Every feature in the app is the same seven files. Once you have seen one, you have seen all of them:

#	LAYER	FILE (INQUIRY FEATURE AS THE EXAMPLE)
1	FE Screen	`components/user-add-inquiry/user-add-inquiry.component.ts + .html`
2	FE Type	`models/architect-inquiry.model.ts`
3	FE HTTP client	`services/architect-inquery.service.ts`
4	SEC Gate	`config/JwtAuthenticationFilter.java` → `config/SecurityConfig.java`
5	BE Endpoint	`controller/ArchitectInquiryController.java`
6	BE Business rules	`service/ArchitectInquiryServiceImpl.java`
7	DB Persistence	`repository/ArchitectInquiryRepo.java` → `model/ArchitectInquiry.java`

The data crosses two translation boundaries on the way down and the same two on the way back: TypeScript object → JSON (Angular's `HttpClient`), and JSON → Java entity (Jackson). Nothing else transforms the shape, which is why the field names in `architect-inquiry.model.ts` must match the field names in `ArchitectInquiry.java` exactly.

2 System architecture

Three processes on three ports. They share nothing but JSON over HTTP and a JDBC connection — no sessions, no server-side rendering, no shared memory.

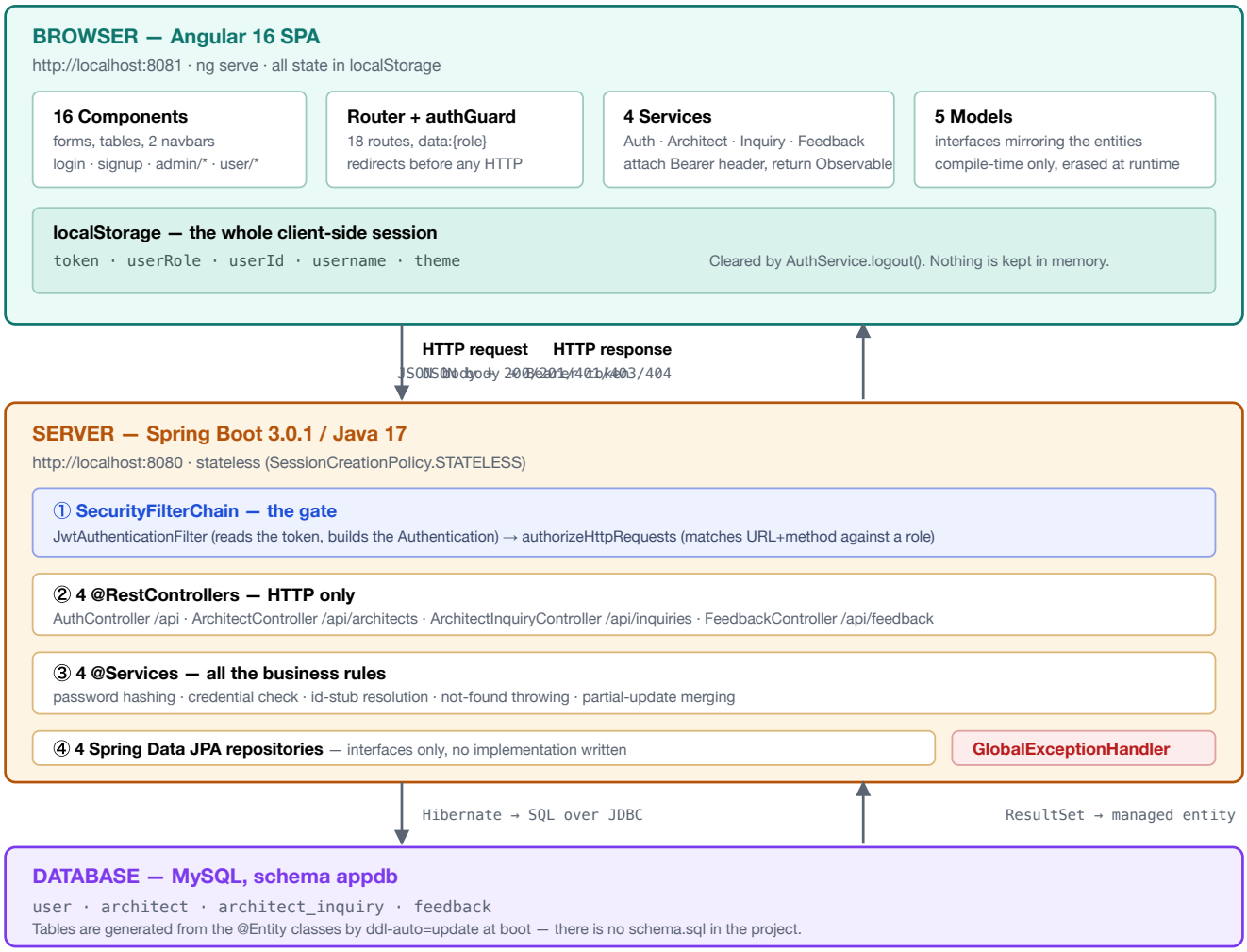


Figure 1 — The three tiers. Each box names the real files; nothing in this diagram is conceptual. Note that the gate (①) sits *in front of* the controllers: by the time any Java method of yours runs, authorisation has already been decided.

Why it is stateless, and what that costs

The server keeps no session. Each request must carry its own proof of identity in the `Authorization` header, which the browser pulls out of `localStorage` and attaches on every call. Three consequences follow, and all three show up in the code:

1. Every service method in Angular builds its own header — `private authHeaders()` appears identically in all three data services. There is no HTTP interceptor; the header is attached by hand per call.
2. The server re-reads the token and re-loads the user from the database on *every single request* (`MyUserDetailsService.loadUserByUsername`). That is one extra `SELECT … WHERE email = ?` per API call — correct, if not the fastest design.
3. Logging out is purely a client-side act: `localStorage.removeItem('token')`. The token itself stays cryptographically valid for its full 24 hours. There is no server-side revocation list.

3 The data model

Four entities, two of which point at the other two. Every relation is `@ManyToOne` and every one is *unidirectional* — deliberately.

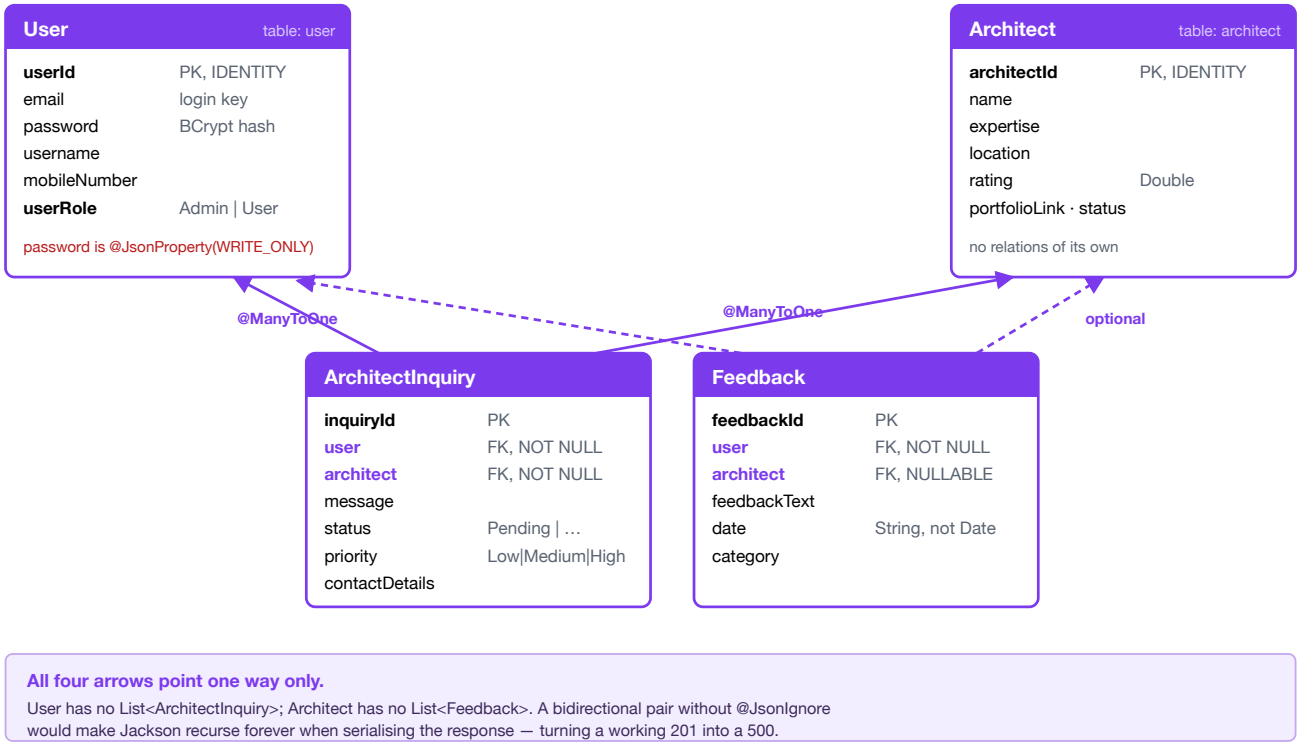


Figure 2 — The entity model. Solid arrows are `nullable=false`; dashed is the one optional relation (feedback need not name an architect). Field names here are byte-identical to the TypeScript interfaces in `angularapp/src/app/models/`.

Two details worth pausing on

THE PASSWORD NEVER COMES BACK OUT

```
// model/User.java
@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private String password;
```

`WRITE_ONLY` means Jackson will *read* the field when deserialising an incoming registration or login body, but will *omit* it when serialising a response. So `POST /api/register` returns the created user with an id and no password hash, and every nested `user` object inside an inquiry or feedback response is similarly scrubbed. One annotation, applied once, protects every endpoint that ever returns a `User`.

THE SCHEMA IS GENERATED, WHICH IS WHY YOU MUST DROP THE DATABASE

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:mysql://localhost/appdb?createDatabaseIfNotExist=true
```

Hibernate creates missing tables and columns at boot but never deletes rows. Because the primary keys are `GenerationType.IDENTITY`, a second run against a non-empty `appdb` starts handing out `userId` 3 and 4 instead of 1 and 2 — and the JUnit suite hard-codes 1 and 2. That is the mechanism behind the project's first hard rule.

DROP THE DATABASE BEFORE EVERY BACKEND RUN

```
mysql -u root --protocol=tcp -p -e "DROP DATABASE IF EXISTS appdb; CREATE DATABASE appdb;"
```

Skipping this makes tests BE-05 onward fail with no code change — the single most common cause of phantom failures in this project.

4 The backend, layer by layer

Each layer has exactly one job and knows nothing about the layer two steps away. Here is what each one is responsible for, and — just as usefully — what it deliberately is not.

Controllers: translate HTTP, and nothing else

Every controller method is two or three lines. It takes a deserialised body or a path variable, calls one service method, and returns the result. It contains no `if`, no `try/catch`, no `ResponseEntity` plumbing, and no security annotations.

```
// controller/ArchitectInquiryController.java
@RestController
@RequestMapping("/api/inquiries")
@CrossOrigin(origins = "*")
public class ArchitectInquiryController {

    @Autowired private ArchitectInquiryService inquiryService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)              // 201, declared not constructed
    public ArchitectInquiry addInquiry(@RequestBody ArchitectInquiry inquiry) {
        return inquiryService.createInquiry(inquiry);
    }

    @GetMapping("/user/{userId}")
    public List<ArchitectInquiry> getInquiriesByUserId(@PathVariable Long userId) {
        return inquiryService.getInquiriesByUserId(userId);
    }
    // … getAll, update, delete — same shape
}
```

Status codes are expressed declaratively: `@ResponseStatus(HttpStatus.CREATED)` for the three create endpoints, the implicit 200 everywhere else, and every failure status produced centrally by `GlobalExceptionHandler`. Nothing in a controller ever decides a status code at runtime.

Services: where the thinking happens

Four interfaces, four `Impl` classes. Three jobs recur:

1. HASH ON THE WAY IN, COMPARE ON THE WAY BACK

```
// service/UserServiceImpl.java
public User createUser(User user) {
    user.setPassword(passwordEncoder.encode(user.getPassword()));   // BCrypt
    return userRepo.save(user);
}

public User loginUser(User user) {
    return userRepo.findByEmail(user.getEmail())
            .filter(found -> passwordEncoder.matches(user.getPassword(), found.getPassword()))
            .orElseThrow(() -> new InvalidCredentialsException("Invalid email or password."));
}
```

Note the `.filter(…).orElseThrow(…)` shape: a wrong password and an unknown email both land in the same branch and produce the same 401 message. That is intentional — it gives an attacker no way to tell which of the two was wrong.

2. RESOLVE ID-ONLY STUBS INTO MANAGED ENTITIES

The browser does not know the full `User` or `Architect` object, and should not have to send one. It sends a stub — `"user": { "userId": 2 }` — which Jackson happily deserialises into a `User` with one field set and every other field null. Saving that object directly would try to INSERT a second, half-empty user row and fail. So the service swaps the stub for the real row first:

```
// service/ArchitectInquiryServiceImpl.java
public ArchitectInquiry createInquiry(ArchitectInquiry inquiry) {
    if (inquiry.getUser() == null || inquiry.getUser().getUserId() == null
            || inquiry.getArchitect() == null || inquiry.getArchitect().getArchitectId() == null) {
        throw new ArchitectInquiryException("An inquiry must name both a user and an architect.");
    }

    Long userId      = inquiry.getUser().getUserId();
    Long architectId = inquiry.getArchitect().getArchitectId();

    // the request sends id-only stubs — resolve them into managed entities
    inquiry.setUser(userRepo.findById(userId)
            .orElseThrow(() -> new ArchitectInquiryException("User with id " + userId + " does not exist.")));
    inquiry.setArchitect(architectRepo.findById(architectId)
            .orElseThrow(() -> new ArchitectInquiryException("Architect with id " + architectId + " does not
exist.")));

    return inquiryRepo.save(inquiry);
}
```

`FeedbackServiceImpl` does the same, with one difference that matches the nullable column: if no architect was named it explicitly sets the relation to `null` rather than leaving a stub in place.

3. MERGE UPDATES FIELD BY FIELD, NEVER WHOLESALE

```
// service/ArchitectInquiryServiceImpl.java
public ArchitectInquiry updateInquiry(Long inquiryId, ArchitectInquiry updated) {
    ArchitectInquiry existing = getInquiryById(inquiryId);   // throws 404 if absent

    existing.setMessage(updated.getMessage());
    existing.setStatus(updated.getStatus());
    existing.setPriority(updated.getPriority());
    existing.setContactDetails(updated.getContactDetails());
    // keep the existing user/architect relations — do not reattach the request's stubs

    return inquiryRepo.save(existing);
}
```

The comment is the important part. An Admin updating an inquiry's status sends back the whole object, stubs included. Copying those stubs onto the managed entity would null out the foreign keys. So the relations are simply left alone: an update can change the four scalar fields and nothing else.

Repositories: four interfaces, zero implementations

```
UserRepo              extends JpaRepository<User, Long>              { Optional<User> findByEmail(String email); }
ArchitectRepo         extends JpaRepository<Architect, Long>         { }
ArchitectInquiryRepo  extends JpaRepository<ArchitectInquiry, Long>  { List<ArchitectInquiry> findByUserUserId(Long
userId); }
FeedbackRepo          extends JpaRepository<Feedback, Long>          { List<Feedback> findByUserUserId(Long userId);
}
```

Spring Data writes the implementations at startup by parsing the method names. `findByUserUserId` decomposes as *find by* → the `user` property → that user's `userId` property, and becomes a join:

```
select i.* from architect_inquiry i
  inner join user u on i.userId = u.userId
 where u.userId = ?
```

Inherited from `JpaRepository` and used throughout: `save` (INSERT or UPDATE depending on whether the id is null), `findById` → `Optional`, `findAll`, `delete`.

Exceptions: one class per status code

EXCEPTION	STATUS	THROWN WHEN
`InvalidCredentialsException`	401	Unknown email, or a password that fails `BCrypt.matches`
`ResourceNotFoundException`	404	`findById` came back empty on an update, delete or get-by-id
`InvalidRequestException`	400	A malformed or incomplete payload
`DuplicateArchitectException`	409	An architect that already exists
`ArchitectInquiryException`	—	Missing or unresolvable `user` / `architect` stub

`@RestControllerAdvice` on `GlobalExceptionHandler` catches these anywhere in the application and renders each as the same small JSON envelope:

```
{ "status": 404, "message": "Inquiry with id 7 does not exist." }
```

5 Security — the JWT lifecycle

A token is minted once at login and re-verified on every subsequent request. The whole mechanism is five small classes, and the order they run in is what determines whether you get a 200, a 401 or a 403.

The five classes

FILE	ROLE IN THE FLOW
`config/JwtUtils.java`	Mints and parses tokens. HS256, a hard-coded secret, a 24-hour expiry, and a `role` claim alongside the email subject.
`config/JwtAuthenticationFilter.java`	Runs once per request, before Spring's own filters. Reads the header, validates, loads the user, populates `SecurityContextHolder`.
`config/MyUserDetailsService.java`	Turns an email into a database row. Called by the filter on every request.
`config/UserPrinciple.java`	Wraps the `User` entity as a `UserDetails` and — critically — derives the authority `"ROLE_" + user.getUserRole()`.
`config/SecurityConfig.java`	The rule table. Maps every URL + HTTP method to a required role.

Minting: what is actually inside the token

```
// config/JwtUtils.java
public String generateToken(String email, String userRole) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("role", userRole);                // ← read from the DB row, never from the request
    return Jwts.builder()
            .setClaims(claims)
            .setSubject(email)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_MS))  // 24h
            .signWith(SignatureAlgorithm.HS256, SECRET)
            .compact();
}
```

Decoded, the payload of a real token from this app looks like:

```
{ "role": "User", "sub": "user@test.com", "iat": 1775260800, "exp": 1775347200 }
```

> **THE ROLE MUST COME FROM THE DATABASE**
>
> A token that omits the role, or copies it from the login request body, still passes both login tests — the client gets a token and a 200. It then fails six later tests for reasons that look entirely unrelated, because the server can no longer tell the two actors apart.

Verifying: the request lifecycle in full

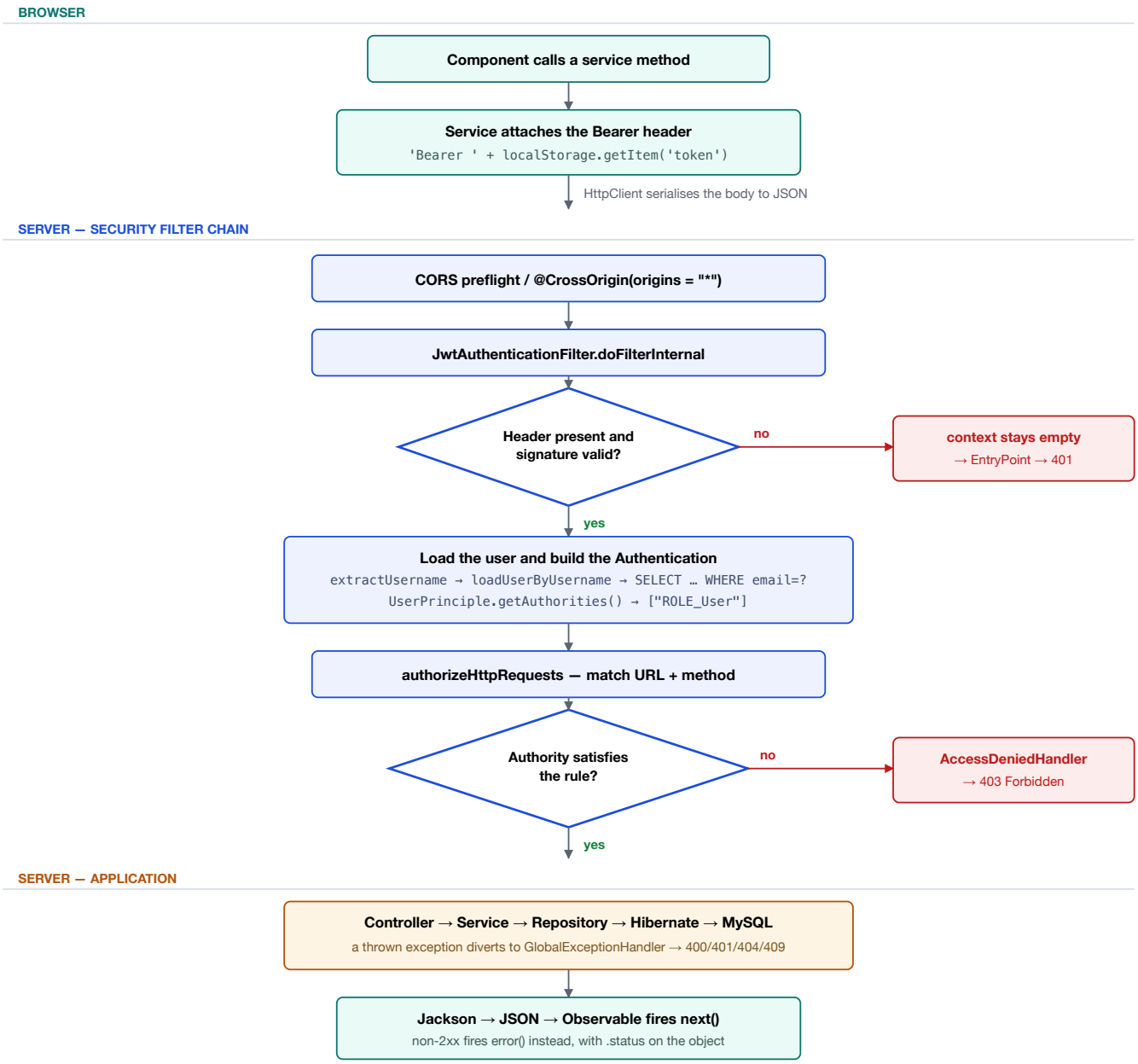


Figure 3 — One request, start to finish. The two diamonds are the only places a request can be rejected before your code runs, and they produce different codes: **401** means "I could not identify you", **403** means "I know exactly who you are and you may not do this".

401 versus 403 — the diagnostic that saves the most time

These two are not interchangeable here, and the graded suite checks for 403 specifically in eleven places. The distinction maps directly onto the two diamonds above:

CODE	REJECTED AT	WHAT IT TELLS YOU
401	Diamond 1 — the filter	No usable identity. Missing header, malformed token, bad signature, or expired. The `SecurityContext` was never populated.
403	Diamond 2 — the rule table	Identity established, role insufficient. A *valid* token for the *wrong* role.

> **GETTING 401 WHERE YOU EXPECT 403**
>
> This almost always means the JWT filter is not populating `SecurityContextHolder` — not that your role rules are wrong. Every "forbidden" assertion in the suite sends a perfectly valid token that simply carries the other role; if those come back 401, authentication broke before authorisation was ever consulted.

The rule table, and why its order matters

```
// config/SecurityConfig.java — excerpt, in evaluation order
.requestMatchers("/api/register", "/api/login").permitAll()
.requestMatchers("/error").permitAll()          // see note below

.requestMatchers(POST, "/api/architects").hasRole("Admin")
.requestMatchers(GET,  "/api/architects").authenticated()      // ← both roles
.requestMatchers(GET,  "/api/architects/*").hasRole("Admin")   // ← by-id is Admin only
.requestMatchers(PUT,  "/api/architects/*").hasRole("Admin")

.requestMatchers(GET,  "/api/inquiries/user/*").hasRole("User")  // most specific FIRST
.requestMatchers(POST, "/api/inquiries").hasRole("User")
.requestMatchers(GET,  "/api/inquiries").hasRole("Admin")
.requestMatchers(PUT,  "/api/inquiries/*").hasRole("Admin")

.requestMatchers(GET,  "/api/feedback/user/*").hasRole("User")
.requestMatchers(POST, "/api/feedback").hasRole("User")
.requestMatchers(GET,  "/api/feedback").hasRole("Admin")

.anyRequest().authenticated()
```

Three rules govern how this block is written, and breaking any of them produces a failure that looks like a bug somewhere else:

1. **Single `*`, never `**`.** `/api/architects/*` matches one path segment. `/api/architects/**` would also match the bare collection URL, so the Admin-only by-id rule would swallow the both-roles list rule, and a User would start getting 403 on `GET /api/architects`.
2. **Most specific first.** Matchers are evaluated top to bottom and the first match wins. If `GET /api/inquiries` (Admin) were declared before `GET /api/inquiries/user/*` (User)… it would not actually catch it — but with `**` patterns it would, which is exactly how this class of bug arrives. Declaring narrow before broad makes the file correct regardless.
3. **`/error` must be `permitAll()`.** When Spring MVC produces a 403, the servlet container re-dispatches internally to `/error`. That re-entry passes through the filter chain again; if `/error` required authentication, the entry point would overwrite the 403 with a 401 and every forbidden-assertion test would fail for a reason nowhere near the role rules.

The role matrix, authoritatively

ENDPOINT	METHOD	ADMIN	USER
`/api/register`, `/api/login`	POST	201 / 200	201 / 200
`/api/architects`	GET	200	200
`/api/architects`	POST	201	403
`/api/architects/{id}`	GET	200	403
`/api/architects/{id}`	PUT	200	403
`/api/inquiries`	POST	403	201
`/api/inquiries`	GET	200	403
`/api/inquiries/user/{id}`	GET	403	200
`/api/inquiries/{id}`	PUT	200	403
`/api/feedback`	POST	403	201
`/api/feedback`	GET	200	403
`/api/feedback/user/{id}`	GET	403	200

WHERE THE SPEC IS WRONG

The SRS PDF (p.43) says `GET /api/feedback` may be `permitAll()` and that security may be commented out for auto-evaluation. Both pieces of advice break the graded suite: test BE-14 requires a User to receive **403** from that endpoint, and eleven tests depend on real role enforcement. Where the PDF and the test files disagree, the test files win.

6 The frontend, layer by layer

A conventional Angular module app: one `NgModule`, one router, one guard, four injectable services, five interfaces, sixteen screens. The interesting design choices are all about where identity is kept.

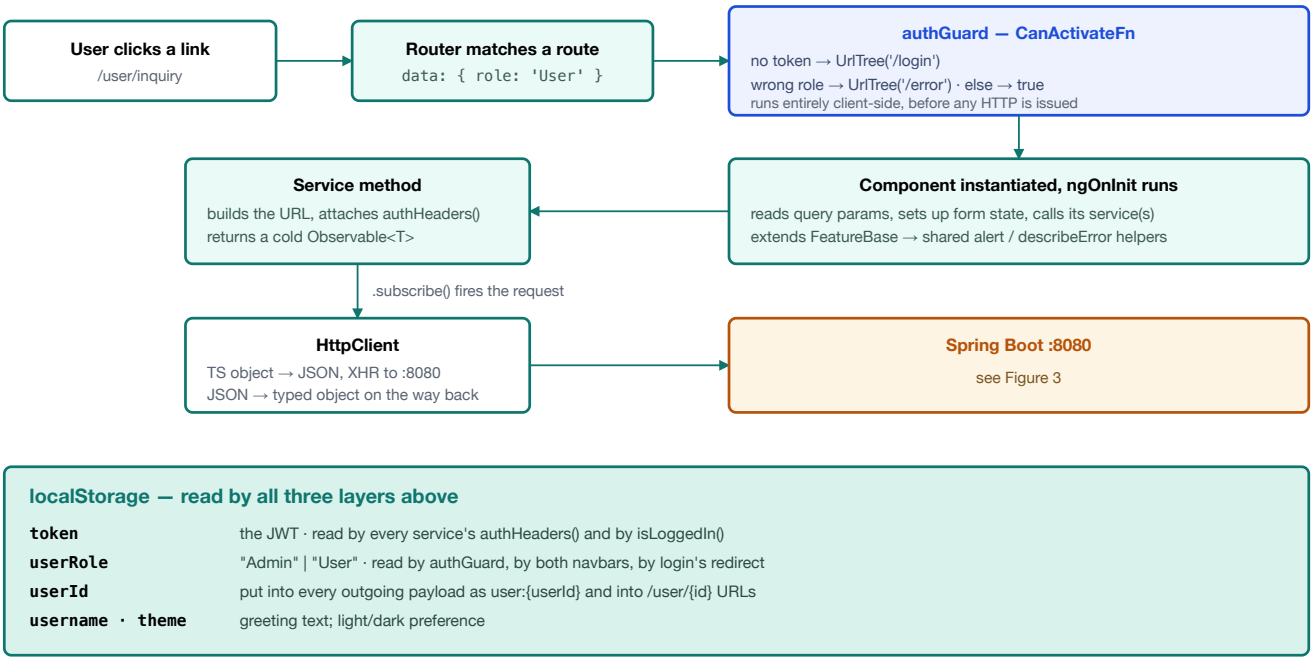


Figure 4 — The client-side flow. `localStorage` is the hub: the guard reads it to decide navigation, the services read it to authenticate, and the components read it to fill in `userId` on outgoing payloads.

AuthService: the only service that writes identity

Login is the one place where the client learns who it is. The `tap` operator in the pipeline is what makes the rest of the app work:

```
// services/auth.service.ts
login(login: Login): Observable<any> {
  return this.http.post<any>(`${this.apiUrl}/api/login`, login).pipe(
    tap(response => {
      if (response && response.token) {
        localStorage.setItem('token', response.token);
        localStorage.setItem('userRole', response.userRole);   // drives authGuard + navbars
        localStorage.setItem('userId', String(response.userId)); // drives every payload
        localStorage.setItem('username', response.username);
        this.userRoleSubject.next(response.userRole);           // BehaviorSubject for live UI
        this.userIdSubject.next(response.userId);
      }
    })
  );
}
```

Alongside the `localStorage` writes there are two `BehaviorSubject`s exposed as `userRole$` and `userId$`, so a component already on screen (a navbar, say) updates reactively rather than waiting for a reload. The synchronous accessors — `isAdmin()`, `isUser()`, `isLoggedIn()`, `getUserId()` — read `localStorage` directly, because the guard needs an answer immediately and cannot wait on a stream.

The guard: a courtesy, not a security boundary

```
// guards/auth.guard.ts
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (!authService.isLoggedIn()) return router.createUrlTree(['/login']);

  const requiredRole = route.data?.['role'];
  if (requiredRole === 'Admin' && !authService.isAdmin()) return router.createUrlTree(['/error']);
  if (requiredRole === 'User'  && !authService.isUser())  return router.createUrlTree(['/error']);

  return true;
};
```

The guard runs in the browser and reads a value the user could edit in devtools. It is not what protects the data — `SecurityConfig` is. What the guard buys you is a better experience: a User who types `/admin/architects` lands on a friendly error page instead of watching a half-rendered admin table fire an HTTP call that comes back 403. Defence in depth, with the real defence on the server.

The three data services are the same service three times

`ArchitectService`, `ArchitectInquiryService` and `FeedbackService` share an identical shape: a public `apiUrl`, a private `authHeaders()`, and one thin method per endpoint returning an `Observable`.

```
// services/architect-inquery.service.ts  ← note the filename spelling
@Injectable({ providedIn: 'root' })
export class ArchitectInquiryService {          // ← class spelled correctly
  public apiUrl = 'http://localhost:8080';

  constructor(private http: HttpClient) { }

  private authHeaders(): HttpHeaders {
    return new HttpHeaders({ Authorization: 'Bearer ' + (localStorage.getItem('token') ?? '') });
  }

  addInquiry(inquiry: ArchitectInquiry): Observable<ArchitectInquiry> {
    return this.http.post<ArchitectInquiry>(`${this.apiUrl}/api/inquiries`, inquiry,
                                            { headers: this.authHeaders() });
  }
  // getAllInquiries · getInquiriesByUserId · updateInquiry · deleteInquiry
}
```

> **THE FILENAME IS MISSPELLED ON PURPOSE**
>
> `services/architect-inquery.service.ts` — "inqu**e**ry" — while the class inside is `ArchitectInquiryService` with the correct spelling. The grader's `karma.sh` checks for that exact filename and prints FAILED without even compiling if it differs. (The *model* file, `architect-inquiry.model.ts`, uses the correct spelling.) Never "fix" this.

Observables are cold

A service method returns an `Observable` and sends nothing. The HTTP request is issued only when a component calls `.subscribe()`, and every component here uses the object form so success and failure are handled side by side:

```
this.inquiryService.addInquiry(inquiry).subscribe({
  next: ()      => { this.isSaving = false; this.showAlert('success', 'Inquiry Sent', '…'); },
  error: (err)  => { this.isSaving = false; this.showAlert('error', 'Failed',
                       this.describeError(err, 'Failed to submit your inquiry.')); }
});
```

Any non-2xx status — the 403s in the matrix included — arrives in `error`, with `err.status` carrying the code. The shared `FeatureBase` class that every feature component extends provides `describeError()` and the alert state, which is why error

handling looks the same on all sixteen screens.

7 Worked example A — Login

A user types credentials and ends up on a role-specific dashboard. Nine hops. This is the only flow that *creates* a token; everything afterwards merely presents it.

Scenario. `user@test.com` / `user123` exists in the `user` table with `userId = 2` and `userRole = 'User'`.

1

FE **The form submits**

`components/login/login.component.html → login.component.ts`

Two-way bindings have filled `loginData`. The template's email field carries an HTML pattern so an obviously malformed address never leaves the browser.

```
loginData = { email: 'user@test.com', password: 'user123' };
this.isLoading = true;
this.authService.login(this.loginData).subscribe({ next: …, error: … });
```

2

FE **AuthService issues the POST**

`services/auth.service.ts`

No auth header here — there is no token yet, and `/api/login` is `permitAll()`.

```
POST http://localhost:8080/api/login
Content-Type: application/json

{ "email": "user@test.com", "password": "user123" }
```

3

SEC **The filter chain waves it through**

`config/SecurityConfig.java → JwtAuthenticationFilter.java`

No `Authorization` header, so the filter does nothing at all and leaves the context empty. The first matcher in the table — `requestMatchers("/api/register", "/api/login").permitAll()` — matches, so an empty context is fine.

4

BE **AuthController receives a deserialised User**

`controller/AuthController.java`

Jackson binds the body to a `User`; `email` and `password` are set, the other four fields are null. `password` is writable because `WRITE_ONLY` blocks only serialisation.

```
@PostMapping("/login")
public LoginDTO login(@RequestBody User loginRequest) {
    User user = userService.loginUser(loginRequest);
    …
}
```

5

BE **UserServiceImpl verifies the credentials**

`service/UserServiceImpl.java`

```
return userRepo.findByEmail(user.getEmail())
        .filter(found -> passwordEncoder.matches(user.getPassword(), found.getPassword()))
        .orElseThrow(() -> new InvalidCredentialsException("Invalid email or password."));
```

`BCryptPasswordEncoder.matches` re-hashes the submitted password using the salt embedded in the stored hash and compares. Either failure — unknown email or wrong password — produces the same 401, so nothing leaks about which.

6

DB **MySQL answers**

`repository/UserRepo.java — derived from the method name`

```
select u.userId, u.email, u.password, u.username, u.mobileNumber, u.userRole
  from user u where u.email = 'user@test.com';

-- 2 | user@test.com | $2a$10$N9qo8uLO… | TestUser | 9876543210 | User
```

7

BE **The controller assembles the response DTO**

`controller/AuthController.java + config/JwtUtils.java`

The role handed to `generateToken` is `user.getUserRole()` — the value that just came out of the database row, not anything the client sent.

```
LoginDTO dto = new LoginDTO();
dto.setToken(jwtUtils.generateToken(user.getEmail(), user.getUserRole()));
dto.setUsername(user.getUsername());
dto.setUserRole(user.getUserRole());
dto.setUserId(user.getUserId());
return dto;                                  // 200 OK
```

The response is a `LoginDTO`, not a `User` — which is how the password hash stays out of the picture entirely, independent of the Jackson annotation.

```
200 OK
{
  "token":    "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiVXNlciIsInN1YiI6InVzZXJAdGVzdC5jb20i…",
  "username": "TestUser",
  "userRole": "User",
  "userId":   2
}
```

THE KEY MUST BE EXACTLY `token`

Eleven JUnit tests read `body.get("token")` from this response. Name it `jwt`, `accessToken` or `authToken` and all eleven fail with a `NullPointerException` that points nowhere near the actual mistake.

8

FE **The tap operator stores the identity**

`services/auth.service.ts`

Four `localStorage` keys are written and two `BehaviorSubject`s are pushed, *before* the component's `next` callback runs. Ordering matters: the component's very next statement asks `isAdmin()`, which reads `localStorage`.

9

FE **The component routes by role**

`components/login/login.component.ts`

```
next: () => {
  this.isLoading = false;
  if (this.authService.isAdmin()) this.router.navigate(['/admin']);   // control panel
  else                            this.router.navigate(['/home']);    // user home
}
```

Navigation re-enters `authGuard`, which now finds a token and a matching role, so it returns `true` and the destination component mounts.

THE SAME FLOW WHEN THE PASSWORD IS WRONG

```
POST /api/login          { "email": "user@test.com", "password": "wrong" }

  → UserServiceImpl: .filter(…) fails  →  throw InvalidCredentialsException
  → GlobalExceptionHandler.handleUnauthorized
  → 401  { "status": 401, "message": "Invalid email or password." }
  → Angular: error callback, err.status === 401
  → LoginComponent: this.errorMessage = 'Invalid Email or Password'
```

The component also special-cases `err.status === 0`, which is not a server reply at all but the browser reporting that it could not reach `:8080` — a dead backend or a CORS rejection. Separating the two saves real debugging time.

8 Worked example B — A user submits an inquiry

The flow that touches every layer in the project: a guarded route, a token-bearing POST, a role check, id-stub resolution, two SELECTs, an INSERT, and a nested JSON response. If you read only one trace, read this one.

Scenario. The user from example A is logged in (`userId = 2`, `userRole = 'User'`). They are browsing `/user/architects` and click "Send Inquiry" on architect #1.

1 **FE** **Navigation, guarded**

`app-routing.module.ts → guards/auth.guard.ts`

The browse screen navigates to `/user/inquiry?architectId=1`. The route declares `canActivate: [authGuard], data: { role: 'User' }`. The guard reads `localStorage.userRole === 'User'` and returns `true`. No HTTP yet.

2 **FE** **ngOnInit pre-selects and loads the dropdown**

`components/user-add-inquiry/user-add-inquiry.component.ts`

```
const preset = this.route.snapshot.queryParamMap.get('architectId');
if (preset) this.architectId = Number(preset);      // → 1
this.loadArchitects();                              // GET /api/architects — allowed for both roles
```

That GET is the single endpoint in the matrix open to both roles, which is exactly why this screen can populate its dropdown without an Admin token.

3 **FE** **save() builds the payload**

`components/user-add-inquiry/user-add-inquiry.component.ts`

`userId` comes from `AuthService`, i.e. from `localStorage`. Both relations are sent as id-only stubs by design.

```
const userId = this.authService.getUserId();          // 2
if (!userId || !this.architectId) { this.showAlert('error', 'Failed', '…'); return; }

// the backend resolves these id-only stubs into managed entities
const inquiry: ArchitectInquiry = {
  user:      { userId },                              // ← stub
  architect: { architectId: this.architectId },        // ← stub
  message:        this.message,
  priority:       this.priority,                       // 'High'
  contactDetails: this.contactDetails,
  status:         'Pending'                            // client-set initial state
};
this.inquiryService.addInquiry(inquiry).subscribe({ next: …, error: … });
```

4 **FE** **The service attaches the token and POSTs**

`services/architect-inquery.service.ts`

```
POST http://localhost:8080/api/inquiries
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiVXNlciIsInN1YiI6InVzZXJAdGVzdC5jb20i…

{
  "user":           { "userId": 2 },
  "architect":      { "architectId": 1 },
  "message":        "I need a modern 3BHK design for a 2400 sqft plot.",
  "priority":       "High",
  "contactDetails": "9876543210",
  "status":         "Pending"
}
```

5

SEC **The JWT filter establishes identity**

`config/JwtAuthenticationFilter.java → MyUserDetailsService → UserPrinciple`

```
header.startsWith("Bearer ")          // yes → token = header.substring(7)
jwtUtils.validateToken(token)         // HS256 signature + expiry → true
jwtUtils.extractUsername(token)       // "user@test.com" (the subject)
userDetailsService.loadUserByUsername("user@test.com")
    → SELECT … FROM user WHERE email = 'user@test.com'      // one extra query, every request
    → new UserPrinciple(user)
    → getAuthorities() = [ SimpleGrantedAuthority("ROLE_User") ]
SecurityContextHolder.getContext().setAuthentication(auth)
```

Note that the role used downstream is derived from the freshly loaded database row, not read out of the token's `role` claim. A role changed in the database therefore takes effect on the very next request, without reissuing the token.

Note also the `catch (Exception e) { SecurityContextHolder.clearContext(); }` wrapper: a malformed or expired token never throws out of the filter. It leaves the context empty and lets the authorisation step decide — which is how you get a clean 401 instead of a 500 stack trace.

6

SEC **The rule table authorises**

`config/SecurityConfig.java`

```
.requestMatchers(HttpMethod.POST, "/api/inquiries").hasRole("User")

// hasRole("User") requires the authority "ROLE_User" — the ROLE_ prefix is
// added by Spring, which is precisely why UserPrinciple builds it by hand.
// authority present → granted → the request proceeds to the controller
```

7

BE **The controller hands off**

`controller/ArchitectInquiryController.java`

```
@PostMapping @ResponseStatus(HttpStatus.CREATED)
public ArchitectInquiry addInquiry(@RequestBody ArchitectInquiry inquiry) {
    return inquiryService.createInquiry(inquiry);
}
```

At this point `inquiry.getUser()` is a `User` object with `userId = 2` and five null fields, and `inquiry.getArchitect()` is an `Architect` with `architectId = 1` and six null fields. These are *detached* objects — unknown to the persistence context.

8

BE **The service resolves the stubs**

`service/ArchitectInquiryServiceImpl.java`

Validate, then swap each stub for the managed entity it refers to:

```
inquiry.setUser(userRepo.findById(2L).orElseThrow(…));            // SELECT … WHERE userId = 2
inquiry.setArchitect(architectRepo.findById(1L).orElseThrow(…));  // SELECT … WHERE architectId = 1
return inquiryRepo.save(inquiry);
```

SKIP THIS STEP AND THE 201 BECOMES A 500

Saving the detached stub makes Hibernate treat it as a brand-new `User` to insert — a row with a non-null id but null `email` and null `password` — and the INSERT fails against the existing primary key. The error surfaces as a `TransientPropertyValueException` or a constraint violation, neither of which mentions the real cause. Resolving nested payloads from the repositories before saving is mandatory for both inquiries and feedback.

9

DB **Hibernate writes the row**

`generated — visible in the log because spring.jpa.show-sql=true`

```
select u.* from user u where u.userId = 2;
select a.* from architect a where a.architectId = 1;

insert into architect_inquiry (architectId, contactDetails, message, priority, status, userId)
values (1, '9876543210', 'I need a modern 3BHK design for a 2400 sqft plot.', 'High', 'Pending', 2);

-- generated key: inquiryId = 1
```

The id is assigned by MySQL's `AUTO_INCREMENT` and read back into the entity, which is what makes it available in the response.

10

BE **Jackson serialises the saved entity**

`model/ArchitectInquiry.java · model/User.java`

The returned object now holds the *fully populated* user and architect, because the service replaced the stubs. So the response is richer than the request — and the password is absent, courtesy of `WRITE_ONLY`.

```
201 Created
{
  "inquiryId": 1,
  "user": {
    "userId": 2, "email": "user@test.com", "username": "TestUser",
    "mobileNumber": "9876543210", "userRole": "User"
  },
  "architect": {
    "architectId": 1, "name": "John Architect", "expertise": "Residential",
    "location": "Mumbai", "rating": 4.5,
    "portfolioLink": "http://portfolio.com/john", "status": "Active"
  },
  "message":        "I need a modern 3BHK design for a 2400 sqft plot.",
  "status":         "Pending",
  "priority":       "High",
  "contactDetails": "9876543210"
}
```

This is also where unidirectional relations earn their keep. Had `User` carried a `List<ArchitectInquiry>`, Jackson would serialise the inquiry → its user → that user's inquiries → each inquiry's user → … until the stack overflowed, and the 201 would arrive as a 500.

11

FE **The component confirms and redirects**

`components/user-add-inquiry/user-add-inquiry.component.ts`

```
next: () => {
  this.isSaving = false;
  this.showAlert('success', 'Inquiry Sent', 'Your inquiry has been successfully submitted.');
}

override closeAlert(): void {
  const wasSuccess = this.alert.type === 'success';
  this.alert.show = false;
  if (wasSuccess) this.router.navigate(['/user/inquiries']);   // go see it in the list
}
```

Dismissing a *successful* alert navigates to the list screen; dismissing an error alert leaves the form intact so the text can be corrected and resubmitted. One override, two behaviours.

9 Worked example C — Admin lists all inquiries

A read path. Shorter than the write, and it shows the derived-query join and the role inversion in the same trace.

Scenario. An Admin (`userId = 1` , `userRole = 'Admin'`) opens `/admin/inquiries` to triage what users have sent.

1

FE **Guard, then ngOnInit**

`app-routing.module.ts · admin-view-inquiry.component.ts`

The route carries `data: { role: 'Admin' }` ; the guard sees `isAdmin() === true` and allows it. `ngOnInit` calls `inquiryService.getAllInquiries()` .

2

FE **The GET goes out**

`services/architect-inquery.service.ts`

```
GET http://localhost:8080/api/inquiries
Authorization: Bearer <admin token, role claim "Admin">
```

3

SEC **Authenticate, then authorise**

`JwtAuthenticationFilter → SecurityConfig`

```
subject "admin@test.com" → DB row → authority ["ROLE_Admin"]

// matcher order matters here. The narrower rule is declared first and
// does not match this URL; the second one does:
.requestMatchers(GET, "/api/inquiries/user/*").hasRole("User")   // no match (no extra segments)
.requestMatchers(GET, "/api/inquiries").hasRole("Admin")         // match → granted
```

Had these been written with `**` instead of `*` , the first rule would have captured the bare collection URL too and the Admin would have received a 403 from an endpoint that is supposed to be theirs.

4

BE **Controller → service → repository**

`ArchitectInquiryController → ArchitectInquiryServiceImpl → ArchitectInquiryRepo`

```
getAllInquiries()  →  inquiryRepo.findAll()      // inherited from JpaRepository
```

5

DB **One query, plus one per relation**

```
select i.* from architect_inquiry i;

-- @ManyToOne defaults to EAGER, so each row's relations are fetched too:
select u.* from user u where u.userId = ?;
select a.* from architect a where a.architectId = ?;
```

This is the classic N+1 pattern — fine at assessment scale, and the thing you would attack first (with `@EntityGraph` or a fetch join) if this list ever had to serve thousands of rows.

6

BE **A JSON array comes back**

```
200 OK
[
  { "inquiryId": 1,
    "user":      { "userId": 2, "email": "user@test.com", "username": "TestUser", … },
    "architect": { "architectId": 1, "name": "John Architect", … },
    "message": "I need a modern 3BHK design…", "status": "Pending", "priority": "High",
    "contactDetails": "9876543210" }
]
```

7

FE **The table renders**

`admin-view-inquiry.component.html`

The array is assigned to a field and an `*ngFor` renders the rows. Because the nested objects came back fully populated, the template can show `inquiry.user.username` and `inquiry.architect.name` directly — no second round trip to resolve names.

The mirror image: a user lists their own

Same screens, same services, opposite role — and a different repository method:

```
GET http://localhost:8080/api/inquiries/user/2
Authorization: Bearer <user token>

  → SecurityConfig: .requestMatchers(GET, "/api/inquiries/user/*").hasRole("User")   ✓
  → ArchitectInquiryController.getInquiriesByUserId(2)
  → inquiryRepo.findByUserUserId(2)
  → select i.* from architect_inquiry i inner join user u on i.userId = u.userId where u.userId = 2
  → 200, only that user's rows
```

The `2` in the URL comes from `localStorage.userId`. Worth noting as a design limitation: the server authorises the *role*, not the *ownership* — nothing in `SecurityConfig` or the service checks that the `{userId}` in the path is the caller's own. Any logged-in User could request `/api/inquiries/user/3`. Closing that gap would mean comparing the path variable against the authenticated principal inside the service; it is out of scope for the graded tests, but it is the first thing to fix if this app ever went near production.

10 Worked example D — A 403 in slow motion

Eleven of the fifteen backend tests assert a refusal. Understanding exactly where the request dies — and what never runs — is what makes those tests debuggable.

Scenario. A User sends a perfectly valid, correctly signed, unexpired token to `POST /api/architects` — an Admin-only endpoint.

1 **FE** **The request**

```
POST http://localhost:8080/api/architects
Authorization: Bearer <valid USER token>

{ "name": "Hacker Architect", "expertise": "Residential", "location": "Mumbai",
  "rating": 4.0, "portfolioLink": "http://x.com", "status": "Active" }
```

2 **SEC** **Authentication SUCCEEDS**

`config/JwtAuthenticationFilter.java`

This is the step people expect to fail, and it does not. The signature verifies, the user loads, the context is populated with `ROLE_User`. The server knows precisely who the caller is.

3 **SEC** **Authorisation FAILS**

`config/SecurityConfig.java`

```
.requestMatchers(HttpMethod.POST, "/api/architects").hasRole("Admin")

  required:  ROLE_Admin
  present:   ROLE_User
  → AccessDeniedException → AccessDeniedHandler → 403 Forbidden
```

4 **SEC** **The internal /error re-dispatch**

`config/SecurityConfig.java — the permitAll("/error") line`

The container re-dispatches to `/error` to render the response body. That re-entry passes through the filter chain a second time. Because `/error` is `permitAll()`, the 403 survives. Without that line, the entry point authenticates the re-dispatch, fails, and overwrites the 403 with a **401** — and every forbidden test in the suite fails for a reason that has nothing to do with the role rules.

5 × **Nothing in the application layer ever ran**

No controller method was invoked. No service method, no repository call, no SQL. The database is untouched and no architect row was created. This matters when you are debugging: if you see your service's log lines for a request that returned 403, the 403 came from somewhere else entirely — not from `SecurityConfig`.

6 **FE** **The client sees an error**

```
403 Forbidden

→ Angular: the error callback fires, err.status === 403
→ FeatureBase.describeError(err, fallback) → an alert on screen
```

In normal use a User never reaches this code path at all, because `authGuard` redirects them to `/error` long before any admin screen can mount and issue the call. The 403 is the server-side backstop for someone bypassing the UI — which is exactly how the JUnit suite talks to the API.

Five responses, five different meanings

STATUS	PRODUCED BY	MEANS
401	`JwtAuthenticationEntryPoint`	No usable token. Missing, malformed, wrong signature, or expired.
401	`GlobalExceptionHandler`	Login specifically: the email or password did not match.
403	`AccessDeniedHandler`	Valid token, wrong role for this URL + method.
404	`GlobalExceptionHandler`	Authorised fine, but the id does not exist.
500	Unhandled	A bug: an unresolved id-stub, or Jackson recursion from a bidirectional relation.
0	The browser, not the server	Backend down, or CORS blocked the call. No HTTP response at all.

11 Complete API reference

Twelve graded endpoints plus three deletes. Each row names the controller method, the service method and the repository call behind it.

Authentication — `AuthController`, base `/api`

ENDPOINT	ROLES	REQUEST → RESPONSE
`POST /register`	public	`User` (email, password, username, mobileNumber, userRole) → **201** + the created `User`, password omitted. → `userService.createUser` → BCrypt → `userRepo.save`
`POST /login`	public	`{ email, password }` → **200** + `LoginDTO { token, username, userRole, userId }`; **401** on mismatch. → `userService.loginUser` → `findByEmail` + `matches` → `jwtUtils.generateToken`

Architects — `ArchitectController`, base `/api/architects`

ENDPOINT	ROLES	REQUEST → RESPONSE
`POST /`	Admin / 403	`Architect` → **201** + saved entity with a generated `architectId`. → `addArchitect` → `architectRepo.save`
`GET /`	Admin + User	— → **200** + `Architect[]`. The only shared endpoint; it feeds both the browse screen and the inquiry dropdown. → `getAllArchitects` → `findAll`
`GET /{id}`	Admin / 403	— → **200** + one `Architect`; **404** if absent. Used to prefill the edit form. → `getArchitectById` → `findById().orElseThrow`
`PUT /{id}`	Admin / 403	`Architect` → **200** + updated entity; **404** if absent. → `updateArchitect` → load, copy fields, `save`
`DELETE /{id}`	Admin / 403	— → **200**. Ungraded.

Inquiries — `ArchitectInquiryController`, base `/api/inquiries`

ENDPOINT	ROLES	REQUEST → RESPONSE
`POST /`	403 / User	`ArchitectInquiry` with id-stubs for user and architect → **201** + entity with both relations fully populated. → `createInquiry` → validate → resolve both stubs → `save`
`GET /`	Admin / 403	— → **200** + every inquiry, each with nested user and architect. → `getAllInquiries` → `findAll`
`GET /user/{id}`	403 / User	— → **200** + that user's inquiries only. → `getInquiriesByUserId` → `findByUserUserId` (derived join)
`PUT /{id}`	Admin / 403	`ArchitectInquiry` → **200**. Merges **only** message, status, priority and contactDetails; relations are deliberately left untouched. → `updateInquiry` → load, set 4 fields, `save`
`DELETE /{id}`	403 / User	— → **200**; **404** if absent. Ungraded.

Feedback — `FeedbackController`, base `/api/feedback` (singular)

ENDPOINT	ROLES	REQUEST → RESPONSE
`POST /`	403 / User	`Feedback` with a user stub and an *optional* architect stub → **201**. → `createFeedback` → resolve user; resolve architect or set null → `save`
`GET /`	Admin / 403	— → **200** + all feedback. The SRS says this may be `permitAll()`; the tests require 403 for a User. → `getAllFeedback` → `findAll`
`GET /user/{id}`	403 / User	— → **200** + that user's feedback only. → `getFeedbackByUserId` → `findByUserUserId`
`GET /{id}`	authenticated	— → **200** + one `Feedback`; **404** if absent. Ungraded.
`DELETE /{id}`	403 / User	— → **200**. Ungraded.

> **THE PATH IS `/api/feedback`, SINGULAR**
>
> Architects and inquiries are plural; feedback is not. It is asserted this way in the test suite, so the inconsistency is load-bearing. Every URL in this document is spelled exactly as the code and the tests have it.

12 Screen map

Sixteen screens. Two public, one shared, six Admin, five User, plus the error page and the two navbars. Each one is a thin shell over one or two service calls.

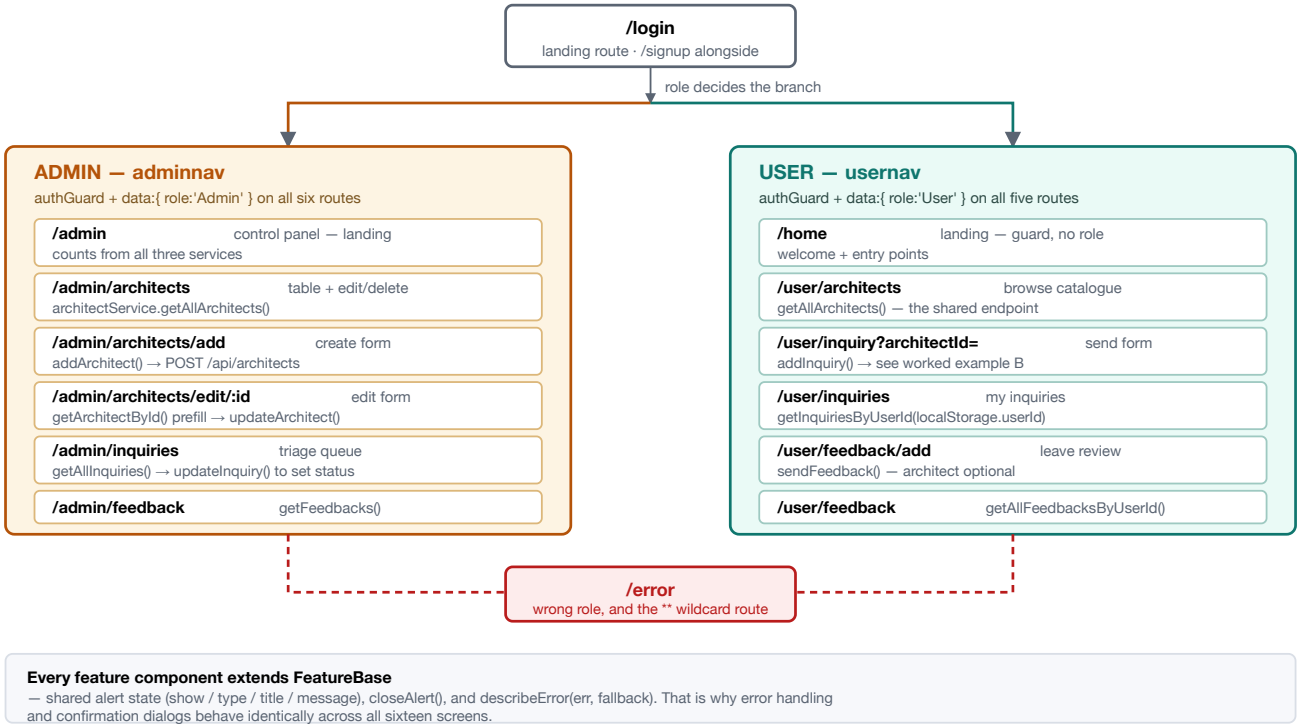


Figure 5 — Screen map. Dashed red arrows are guard redirects, not navigation the user can choose. The `'**'` catch-all route also lands on `/error`.

13 The grading harness

24 automated tests decide the score: 15 Maven/JUnit against the live backend, 9 Karma against the compiled frontend. They are the source of truth — above the SRS PDF, and above anything in this document.

Backend — 15 JUnit tests

These are not unit tests. They boot the whole application and talk to it over HTTP exactly as the browser does, which is why they catch security misconfiguration that a mocked test would miss. They are `@Order`-ed and share state: the two login tests stash JWTs in fields that the following thirteen read.

#	TEST	ASSERTS
01	`backend_testRegisterAdmin`	201, and `userId` 1
02	`backend_testRegisterUser`	201, and `userId` 2
03	`backend_testLoginAdmin`	200 and a non-null `token`; stores the admin JWT
04	`backend_testLoginUser`	200 and a non-null `token`; stores the user JWT
05	`backend_testAdminAddArchitect`	Admin POST → 201, `architectId` 1; User POST → 403
06	`backend_testGetAllArchitects`	200 for *both* roles — the one shared endpoint
07	`backend_testGetArchitectByIdWithRoleValidation`	Admin 200, User 403
08	`backend_testUpdateArchitectWithRoleValidation`	Admin 200, User 403
09	`backend_testUserCanAddArchitectInquiry`	User POST → 201, `inquiryId` 1; Admin POST → 403
10	`backend_testRoleBasedAccessForViewingAllInquiries`	Admin 200, User 403
11	`backend_testUserCanViewOwnInquiries`	`/user/2`: User 200, Admin 403
12	`backend_testAdminCanUpdateInquiry_UserCannot`	Admin PUT 200, User PUT 403
13	`backend_testAddFeedbackWithRoleValidation`	User POST → 201; Admin POST → 403
14	`backend_testGetAllFeedbackWithRoleValidation`	Admin 200, User **403** — contradicts the SRS; the test wins
15	`backend_testGetFeedbackByUserIdWithRoleValidation`	User 200, Admin 403

Eleven of the fifteen contain a 403 assertion. That is the shape of this assessment: it is a security-configuration exercise wearing a CRUD costume.

> **TWO CONSEQUENCES OF THE SHARED, ORDERED STATE**
>
> **Reset the database first.** Tests 01, 02, 05 and 09 assert generated ids of 1 and 2. `ddl-auto=update` keeps old rows, so a second run hands out 3 and 4 and everything from test 05 onward fails with no code change.
>
> **You cannot run a later test alone.** It will fail on `assertNotNull(admintoken)` because tests 03 and 04 never ran. Always include them:
>
> ```
> mvn -q test -
> Dtest=SpringappApplicationTests#backend_testLoginAdmin+backend_testLoginUser+backend_testAdminAddArchitect
> ```

Frontend — 9 Karma tests

These are deliberately shallow. Five instantiate a model object; four inject a service and assert it is truthy. Not one of them exercises a component or a template.

```
// the shape of all four service specs
TestBed.configureTestingModule({ imports: [HttpClientTestingModule] });
service = TestBed.inject(ArchitectInquiryService);
fit('frontend_should_create_architect_inquiry_service', () => {
  expect((service as any)).toBeTruthy();
});
```

TEST	WHAT IT REALLY CHECKS
`frontend_User_model_should_create_an_instance`	That the interface exists, is exported under the expected name, and that an object literal with the expected fields type-checks
`frontend_Login_model_should_create_an_instance`	
`frontend_Architect_model_should_create_an_instance`	
`frontend_Architect_Inquiry_model_should_create_an_instance`	
`frontend_Feedback_model_should_create_an_instance`	
`frontend_should_create_auth_service`	That the class is `@Injectable`, resolvable from the root injector, and that its `HttpClient` dependency injects cleanly
`frontend_should_create_architect_service`	
`frontend_should_create_architect_inquiry_service`	
`frontend_should_create_feedback_service`	

ONE TYPESCRIPT ERROR ANYWHERE FAILS ALL NINE

Karma compiles the whole project before running anything. A typo in a component that no spec even mentions takes down the five model tests too. The practical discipline: re-run `npm test` after adding each component, not after adding ten.

Two related rules — don't edit `angularapp/karma.conf.js` (the grader overwrites it from `karma/karma.conf.js`), and don't rename `services/architect-inquery.service.ts` (`karma.sh` greps for that exact misspelling and prints FAILED without compiling).

Why the split is 15/9

Worth internalising, because it tells you where to spend effort. All the backend tests assert *behaviour* — real HTTP, real status codes, real role enforcement. All the frontend tests assert *existence* — that classes and interfaces are declared and wireable. Components, templates and styling are entirely ungraded. They are still worth building for a working app, but not one of the 24 points depends on them.

14 Running it, and the traps

Commands

```
# ---- BACKEND: always reset the database first ----
mysql -u root --protocol=tcp -p -e "DROP DATABASE IF EXISTS appdb; CREATE DATABASE appdb;"

cd springapp && mvn -q test          # the 15 graded backend tests
cd springapp && mvn spring-boot:run   # serve on :8080

# ---- FRONTEND ----
nvm use 20
cd angularapp && npm install && npm test    # the 9 graded frontend tests
cd angularapp && npm start                   # serve on :8081
```

To exercise the app by hand, the backend must be running first — the Angular services point at a hard-coded `http://localhost:8080`, so a dead backend shows up as `err.status === 0`, which `LoginComponent` reports as a connection problem rather than bad credentials.

Symptom → cause

SYMPTOM	CAUSE	WHERE TO LOOK
Tests 05+ fail; earlier ones pass; no code changed	Dirty `appdb` — generated ids are no longer 1 and 2	Drop and recreate the schema
401 where a test expects **403**	The JWT filter is not populating `SecurityContextHolder`	`JwtAuthenticationFilter`
All forbidden tests return 401	`/error` is not `permitAll()`, so the re-dispatch overwrites the 403	`SecurityConfig`
User gets 403 on `GET /api/architects`	`**` used instead of `*` — the by-id rule swallowed the collection rule	`SecurityConfig` matchers
Login passes, six later tests fail oddly	The token carries no role, or a role not read from the database row	`JwtUtils.generateToken`, `AuthController`
`NullPointerException` in 11 tests	The login response key is not exactly `token`	`LoginDTO`
POST inquiry/feedback returns **500**	Id-only stubs were saved without being resolved from the repositories	the `…ServiceImpl.create…` methods
500 with a stack overflow on a 201 path	A bidirectional relation without `@JsonIgnore` — Jackson recursion	the `model/` entities
All 9 Karma tests fail, including model tests	One TypeScript compile error somewhere in the project	the full `ng test` output, not the spec
Karma prints FAILED without compiling	`architect-inquery.service.ts` was renamed to the correct spelling	the filename
Frontend shows `status: 0`	Backend not running, or CORS blocked the call	`:8080`, and `@CrossOrigin` / `CrosConfig`

The seven hard rules, in one place

1. **Reset `appdb` before every backend run.** The #1 cause of phantom failures.

2. **Keep Spring Security enabled.** The SRS says you may comment it out for auto-evaluation; that advice fails eleven tests.
3. **Never delete or rename** any existing file, exported symbol, field name or endpoint URL — each is asserted somewhere.
4. **Don't modify `pom.xml`.** Every dependency is already present.
5. **Don't edit `angularapp/karma.conf.js`.** The grader overwrites it.
6. **Spring Boot 3 means `jakarta.*`, never `javax.*`**; and Spring Security 6 means a `SecurityFilterChain` bean with the lambda DSL — `WebSecurityConfigurerAdapter` no longer exists.
7. **Never `git push` or open a PR** in this repository.

What is left, and what it is worth

The scored work is finished: 24 of 24 tests pass. Three things remain open, and none of them carries points — the four empty `exceptions/` stubs, the extra SRS finder methods (STEP-11 sections B and C), and the Phase-2 Gemini feature (section D). If you do pick any of them up, re-run both suites afterwards: the frontend suite in particular fails as a block on a single compile error, so an ungraded component can cost you nine graded points.